

3.1 ADTs

Sunday, 8 October 2023 12:49 PM

Key knowledge and skills	• the motivation for using ADTs • signature specifications of ADTs using operator names, argument types and result types • specification and uses of the following ADTs: • set, list, array, dictionary (associative array) • stack, queue, priority queue • modularisation and abstraction of information representation with ADTs • explain the role of ADTs for data modelling • read and write ADT signature specifications • use ADTs in accordance with their specifications
--------------------------	---

An ADT is an abstract representation of a data structure.

- Store data
- Manipulate data
- Coherent and communicable representation of assumed programming structures

For example, the priority queue ADT assumes a model that automatically sorts elements upon input which aids in the design of algorithms by providing a framework from which to work. No different to assuming that strings exist, etc. Abstraction allows high-level ideas to be communicated without getting bogged down in specifics.

Which is the point of pseudocode!

It is specified from a user's point of view, rather than an implementor's.

Typically, ADT signature specification includes:

- Type dependencies
- Name
- Set of signatures with operator names, argument and result types.

Signatures for ADTs

- o Creation of ADT
- o Addition/removal/manipulation of elements
- o Retrieval of elements
- o ADT meta-variables (attribute)

VCAA custom signature examples

Question 2 (5 marks)

Alex is developing a computer simulation to model a chemical reaction. In the simulation, the reactor is initially empty and the reaction starts when one or more chemicals are added. More chemicals may be added while the reaction is in progress. A chemical reaction is characterised by what chemicals are added, how much of them are added and when they are added.

- a. To store a description of a chemical reaction for simulation, Alex needs the following information about each addition into the reactor:
- the name of the chemical added
 - the time the chemical was added, in seconds, after the reaction begins
 - the amount of the chemical added, in milligrams

Describe a combination of abstract data types (ADTs) that Alex could use to store a description of a chemical reaction. Justify your answer.

3 marks

- Dictionary of dictionaries
- Each inner dictionary can have key:value pairs for name, time, and amount. The keys to the outer dictionary can be indexed by a unique id.
- This allows an arbitrary number of chemicals to be added, and dictionaries allow accessing of any part of the reaction data.

- b. Alex intends to use the combination of ADTs described in part a. often and decides to define the combination of ADTs as a new ADT called ChemEvents.

For the ChemEvents ADT, as described in part a., write the signature specification of the following operations:

- Add a new chemical to the reaction.
- Look up what chemical was added to the reaction at a given time.

2 marks

- Add: chem_events x string x int x int --> chem_events
- Get: int x chemEvents --> string

Specifications

List	Integer indexed Resizable	Create: --> list Append/prepend: element x list --> list Remove: int x list --> list First/last: list --> elem Nth: list x int --> elem Update: list x int x elem --> list
Array	Integer indexed Non-resizable	Create: int --> array N/A due to fixed size Update: array x int x elem --> array Get: array x int --> elem
Set	Not indexed Resizable Unique elements	Create: --> set Insert: element x set --> set Remove: element x set --> set N/A due to unordered nature all: set --> list (?)
Dictionary	Key indexed Resizable Keys form a set to ensure 1-1	Create: --> dict Insert: element x dict x key --> dict Remove: dict x key --> dict Update: element x dict x key --> dict Get: key x dict --> element Keys: dict --> set
Queue	FIFO list	Create: --> queue Enqueue: element x queue --> queue Dequeue: queue --> queue Peek: queue --> elem
Stack	LIFO list	Create: --> stack Push: element x stack --> stack Pop: stack --> stack Peek: stack --> element
Priority queue	Priority-based list	CreateMax: --> maxPrioQueue CreateMin: --> minPrioQueue Enqueue: element x int x pqueue --> pqueue Dequeue: pqueue --> pqueue Peek: pqueue --> elem

+ some attributes to be checked for (almost) all:

isIn
Length
isEmpty

Sample pseudocode:

```
c = new queue
c.enqueue(9)
c.dequeue
c.isEmpty
c.enqueue(8)
c.front
```

In the words of the inimitable Dijkstra:

Are you quite sure that all those bells and whistles, all those wonderful facilities of your so-called "powerful" programming languages belong to the solution set rather than to the problem set?

- Dijkstra (1976) [A Discipline of Programming](#), Prentice-Hall, 1976, p. xiv.

The effective exploitation of his powers of abstraction must be regarded as one of the most vital activities of a competent programmer. The purpose of abstracting is *not* to be vague, but to create a new semantic level in which one can be absolutely precise.

- 1972 [Turing Award](#) Lecture^[1], [Communications of the ACM](#) 15 (10), October 1972: pp. 859–866.

From <https://en.wikiquote.org/wiki/Edsger_W._Dijkstra>

3.1 Graphs

Sunday, 8 October 2023 12:51 PM

Key knowledge and skills	- graphs, including undirected and directed graphs and unweighted and weighted graphs - features of graphs, including paths, weighted path lengths, cycles and subgraphs - categories of graphs, including complete graphs, connected graphs, directed acyclic graphs and trees, and their properties - the structure of decision trees and state graphs - identify and describe properties of graphs - apply ADTs to model real-world problems by selecting an appropriate ADT and justifying its suitability - model basic network and planning problems with graphs, including the use of decision trees and state graphs
--------------------------	--

Graph signature

```

Create: --> graph
(can specify digraph, etc.)

AddNode: graph x node --> graph
AddEdge: graph x node x node --> graph
RmNode: graph x node --> graph
RmEdge
getNode: graph --> set
getEdges: graph --> set
SetNodeAttribute
GetNodeAttribute
SetEdgeAttribute
GetEdgeAttribute
NumVertex: graph --> int
NumEdge
NeighbourNodes: graph x node --> set
Incident
Outgoing
Incoming
AdjacentEdges
...
Too many!!!

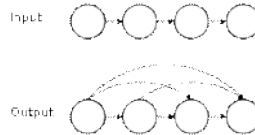
```

A graph is a set of nodes and edges. Edges connect nodes. The size is denoted $|E|$ and the order is $|V|$

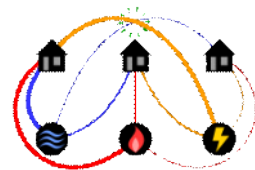
Boolean graph characteristics

Cyclic	Contains a cycle?
Complete	(Of an undirected graph) For every pair of nodes (u,v) does an edge u-v exist?
Connected	(Of undirected) Path between every pair of nodes? (Of directed) Connectivity of a structurally isomorphic graph where edges are undirected.
Directed	... what do i say
Hamiltonian (path/circuit)	Path/circuit that visits each node exactly once? TSP
Eulerian (path/circuit)	Path/circuit that visits each edge exactly once?
K-Colourable (subcase: bipartite)	Can k colours be used to colour the graph such that no two neighbouring nodes share a colour? Graph colouring
Transitive closure	(of two nodes) Is there a path u-v? (of a graph) Typically stored as a boolean matrix of all pairwise transitive closures. $O(1)$ time access.
Planar	Can the graph be represented on a plane such that no edges overlap? (If and only if it does not contain $K_{3,3}$ or K_5 as subgraphs)

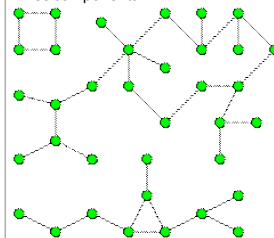
Transitive closure



$K_{3,3}$



Three components



Integer graph characteristics

Diameter	Longest shortest path between two nodes See: Floyd Warshall
Largest clique	Complete subgraph
Chromatic number	minimum number of colours needed to colour a graph See: graph colouring
No. components	A component is a connected subgraph not part of a larger connected subgraph

Other features/definitions

Bridge	Edge between two otherwise disconnected subgraphs
Isolated node	node with no incident edges
Path	Finite sequence of edges which joins a sequence of vertices. Walk: any path Trail: euler Path: hamil.
Circuit	Path which starts and ends on the same node
Subgraph	Subset of vertices/edges of graph.
Loop	Self-edge (not part of SD)
Source	(of a digraph) Node with only outgoing edges
Sink	(of a digraph) Node with only incoming edges
Isomorphic	(of two graphs) Having the same set of vertex/edge relations. I.e., equal.
MST	A subgraph which is a tree and has the lowest possible edge weight cost

Graphs as collections of traits:

- DAG
 - o Directed
 - o Acyclic
- Complete
 - o Undirected
 - o Complete (=> cyclic, connected)
 - o Cyclic
 - o Chromatic number $|V|$
- Tree
 - o Undirected
 - o Acyclic
 - o Connected
 - o Bipartite
- Connected
 - o Connected...
- Arborescence
 - o Directed
 - o Acyclic
 - o Rooted tree structure
 - o 1 path from SOME v to every other node u

The DAG lends itself to a type of sorting called "topological sorting". A node u comes before a node v after topological sorting iff u is a predecessor node of v.

Toposort pseudocode

```

Toposort(g,sorted):
If g.number_of_nodes != 0
  For each source node n in g:
    sorted.append(n)
    g.remove(n)
  Return toposort(g,sorted)
else:
  Return sorted

```

Two applications of graphs are decision trees and state graphs.

Decision tree:

- Rooted tree or directed rooted tree. Model of possible decisions and consequences.

Two applications of graphs are decision trees and state graphs.

Decision tree:

- Rooted tree or directed rooted tree. Model of possible decisions and consequences.
 - o Leaf nodes represent final outcomes
 - o Non-root nodes represent decisions/tests
 - Root node is initial decision/start point
 - o Edges represent outcome of decisions/choices
- Bit like a one-directional flowchart

Decision tree pseudocode

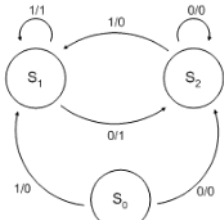
```
decisionTree(g, root)
u = root
While u not leaf node
  For outgoing edge u-v of u:
    If (u-v).condition == true:
      u = v
Return u
```

State graph:

- Graph (typically directed). Describe the behaviour of a system through
 - o a finite number of states (nodes) and
 - o output/transitions/transition conditions (edges)

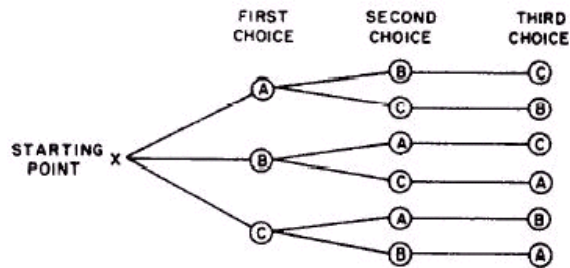
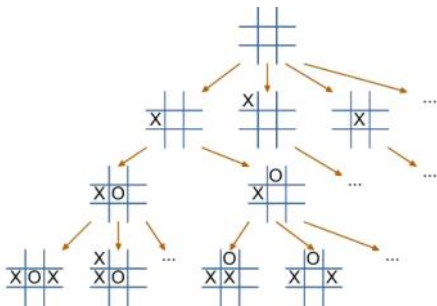
State graph traversal pseudocode

```
stateTraverse(g, initialState)
u = initialState
While not halt:
  For outgoing edge u-v of u
    If (u-v).transition_condition == true
      Do (u-v).action
      u = v
  if u.halt == true
    Halt
```



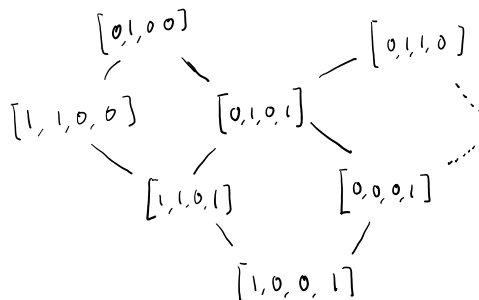
Another type of graph is the "implicit tree", which is usually an arborescence. Basically a combination of state diagram and decision tree. Obvious examples include:

- Tree of recursive calls (a la backtracking)
- Game tree of possible next states
- Order of node discovery by DFS/BFS/other search algorithms
 - o You can find the path to any node by following edges in reverse order



Solution diagrams/ game spaces can also be modelled. Any algorithm that explores "neighbours" uses an implicit graph. For example, hill climbing and simulated annealing.

The "height" of each solution can be based on a heuristic $h(n)$. The knapsack graph yields a 4-regular graph when created based on **Hamming Distance**.



3.2 Algorithmic structures

Sunday, 8 October 2023 12:52 PM

Key knowledge and skills	• basic structure of algorithms • pseudocode concepts, including variables and assignment, sequence, iteration, conditionals and functions • programming language constructs that directly correspond to pseudocode concepts • conditional expressions using the logical operations of AND, OR, NOT • recursion and iteration and their uses in algorithm design • modular design of algorithms and ADTs • interpret pseudocode and execute it manually on given input • write pseudocode • design recursive and iterative algorithms • write modular algorithms using ADTs and functional abstractions
--------------------------	---

What is an algorithm?

A terminating and unambiguous sequence of instructions to solve a problem. The problem of deciding whether a program is an algorithm is undecidable.

Iteration

The process of repeatedly performing the same set of instructions until a given condition.

Recursion *see: Recursion*

The process of a function calling itself until a given base case. The solution to a problem is the same structure as the solution to a subproblem.

Recursion and iteration are isomorphic. However, problems may lend themselves more intuitively to one or the other. Consider the following pseudocode.

<pre>unzipR(lis,out): If lis is a not list: out.append lis return out else: for item in lis out = unzipR(lis,out) return out</pre>	<pre>upzipI(lis): call = new stack for item in lis call.push(item) current = item if current is a list for element in current call.push(element) else out.append(current) return out</pre>
--	--

This code, given a nested list `[a,[b,c],[d,[e]]]` returns a flattened list `[a,b,c,d,e]`. Most would agree that the recursive implementation is a much nicer version, as a nested list is inherently a recursive structure.

Modularisation

The partitioning of algorithms into sub/helper functions, each of which does a subtask that aids in the completion of the problem.

- Aids comprehension
- Abstract away details

Note that the overall runtime complexity of a modular algorithm must depend on the nesting of helper functions.

Boolean algebra refresher: the rules of inference

- $\sim(A \wedge B) = \sim A \vee \sim B$
- $\sim(A \vee B) = \sim A \wedge \sim B$
- $A \wedge (B \vee C) = (A \wedge B) \vee (A \wedge C)$
- $A \vee (B \wedge C) = (A \vee B) \wedge (A \vee C)$

Collatz conjecture program is an example of "Not Necessarily An Algorithm" according to the SD.

```
Collatz(n):
While n not 0
    if 2 | n:
        n = n/2
    else:
        n = 3n+1
```

This is also a good example of the halting problem's undecidability coming into play: if it were decidable, the Collatz conjecture would be trivial!

Let me know if you prove whether it halts and give the solution to me, yeah?

3.2 Algorithms/patterns

Sunday, 8 October 2023 2:10 PM

Key knowledge/skills	• characteristics and suitability of the brute-force search and greedy algorithm design patterns • graph traversal techniques, including breadth-first search and depth-first search • specification, correctness and limitations of the following graph algorithms: • Prim's algorithm for computing the minimal spanning tree of a graph • Dijkstra's algorithm and the Bellman-Ford algorithm for the single-source shortest path problem • the Floyd-Warshall algorithm for the all-pairs shortest path problem and its application to the transitive closure problem • the PageRank algorithm for estimating the importance of a node based on its links • identify and describe recursive, iterative, brute-force search and greedy design patterns within algorithms • design algorithms by applying the brute-force search or greedy algorithm design pattern • select appropriate graph algorithms and justify the choice based on their properties and limitations • use search methods on decision trees and graphs to solve planning problems • implement algorithms, including graph algorithms, as computer programs in a very high-level programming language that directly supports a graph ADT
----------------------	--

Find more details in the "Algorithms" section

time/space complexity and correctness

Decrease and conquer

- Reduce the size of the problem by a fixed amount until it reaches a trivial base case
- Relies on the relationship between the problem and subproblem to be the same
- E.g.,
 - o generating permutations
 - o Search algorithms
 - o Multiplication (repeated addition)

Brute force

- generate/search the entire solution space and pick the best option
- Usually guarantees optimality
- E.g.,
 - o Graph colouring/TSP/knapsack
 - o Floyd-warshall
 - o Generating all possible spanning trees to find the MST
 - o Generating all paths from a node

Note: Even the brute-force algorithm stated in the SD are not mindlessly brute-force. They also employ some form of dynamic programming. E.g., Bellman-Ford and Floyd-Warshall both retain lowest paths found thus far when considering next node.

Greedy

- Select the next best option, assuming that a local best will lead to a global best.
- Quick to run
- Not always return optimal
- E.g.,
 - o Dijkstra, Prim
 - o Coin greedy

Searching/graph traversal has a ubiquitous pattern. BFS uses a queue, so $f(n)$ is in order of discovery. DFS is the opposite, so $f(n)$ has an inverted order. BeFS is short-term heuristic based, and Dijkstra's is cost-based and first compares the tentative with best value. A* is cost + heuristic based, and also first compares tentative with best.

Search(g,start,goal):

visited = new list
next = new list-like
next.insert(start)

while next not empty

 current = remove element from next

 if current is goal or goal fulfilled

 return found!

 else

 append current to visited

 for neighbour in current not in visited

 insert neighbour into next according to $f(n)$

Though the SD probably wants a node to be visited before any subnodes, different types of traversal based on the same model actually allow different orders of traversal too:

1. Traverse left subtree
2. Visit node
3. Traverse right subtree

Gives a different order to

1. Visit node
2. Traverse left subtree
3. Traverse right subtree

((you won't need this info for VCE alg))

Breadth-first search	Searches layer by layer, using a queue ADT to process nodes in order of discovery. Can guarantee shortest path on an unweighted graph.
Depth-first search	Searches using a stack ADT, processing nodes in reverse order of discovery. Backtracking is a form of DFS.
Best-first search	Searches using a priorityQueue ADT where the priority is based on a heuristic $h(n)$ -- an attribute of the node or edge through which it was found.
Tree search	Refers to the use of DFS/BFS on a tree. Typically for DFS, the order of visiting self/left-subtree/right subtree may differ based on what the user wants.
Prim's algo	Generates an MST of a weighted graph by greedily selecting the lowest cost edge $u-v$, where u is in the MST and v is not. Proof relies on the $u-v$ replacement contradiction. (Or induction, which is longer)
Dijkstra's algo	Single source all node shortest path by greedily selecting the lowest cost node to expand at each step. Fails for negative edges, as it assumes that costs CAN ONLY INCREASE. If we know the lower limit for negative edge weights, we can add an integer to make all weights non-negative :)
Bellman-Ford	Single source all node shortest path by iterating through all edges $V-1$ times. The length of the longest possible path is $v-1$ edges.
Floyd-Warshall	Pairwise transitive closure or shortest paths by using a triple-nested for loop of all vertices, and exploring all $a-b-c$ paths. Constitutes the exploration of the entire search space of all paths ... why?
PageRank	Recursive formula repeatedly applied $p(n, i) = \frac{1-d}{ V } + d * \sum_{incoming} \frac{p(node, i-1)}{node.outgoing\ edges}$ Source nodes have edges to every other node created, modelling 100% chance of randomly landing on a page. d is the chance of following a link.

Pseudocode for generating permutations of list ls

```

Permute(ls):
out = new list
For elem in ls
    ls.append([elem])
For partial in out
    for elem in ls
        if elem not in partial
            new = partial with elem appended
            out.append(new)
    out.remove(partial)

((this pseudocode doesn't return anything right now.
Exercise: make it return a list of all permutations.))
    
```

```

PermuteR(ls,out):
If length(out[1]) >= length(ls)
    return out
else
    next = new list
    For p in out
        For elem in ls
            If elem not in p
                n = p with elem appended
                next.append(n)
    return PermuteR(ls,next)
    
```

Generating all permutations can also be treated as a rooted tree starting with a "null" element.

Pseudocode for generating combinations nCr

```

nCr(n,r) //stacks are nice!
out = new stack
For i from 1 to n
    out.push(list with just i)
For i from 1 to r
    top = out.peek
    for j from 1 to out[last element]
        k = partial solution with j appended
        out.push(j)
    out.pop
    
```

```

nCrr(elem,r, curr)
If r <= 0
    Return [curr]
Else
    out = new list
    For i in elem
        newSub = curr with i appended
        newSuper = elem with elements 1 to i removed
        For next in nCrr(new,r-1,newSub)
            out.append(next)
    return out
    
```

For each node u of the tree, add an edge u-v iff v is not on the path from the root to u.

Pseudocode for generating all STs (ordered search)

```

ST(g)
s = random starting node
sg = new graph with vertex s
out = new list
out.append(sg)
while length(out[1]) != |V|:
    for v in sg where u in out and v not
        new = sg
        sg.add_edge(u-v)
        out.append(new)
    out.remove(sg)
    
```

Pseudocode greedy for hill climbing

```

hillClimb(g,current)
best = current
For neighbour n of current
    Best = maxHeight(n, Best)
If best = current
    Return current
Else
    Return hillClimb(g,best)
    
```

3.2 Proof

Sunday, 8 October 2023 2:10 PM

Key...	induction and contradiction as methods for demonstrating the correctness of simple iterative and recursive algorithms • explain the correctness of the specified graph algorithms • demonstrate the correctness of simple iterative or recursive algorithms using structured arguments that apply the methods of induction or contradiction
--------	---

Induction.

By the well-ordering principle of the natural numbers, if some $P(b)$ is true and $n \geq b$ and $P(n) \rightarrow P(n+1)$, then P is true for all $n \geq b$.

Contradiction

Assuming the consistency of the ZFC ;) if we obtain a contradictory result from a set of propositions, then some of them must be false.

Loop invariant

A property of a loop that remains unchanged no matter how many iterations are performed. This property may be a function of the iteration count, but remains true nevertheless. In mathematics, this relates to mathematical objects and their transformations. E.g.,

- > the roots of a graph are invariant under dilations from the x-axis.
- > the degree of a polynomial is invariant under linear changes to its variables
- > the last k elements are correctly invariant after the k th iteration of the main loop in bubble sort -- i.e., the loop invariant is the number of correct elements

Loop invariants are good for induction proofs!

Remember to prove termination!!!

Induction Loop invariant $P(n)$: Statement to prove Base case: Trivial with $n = 1$ etc. $P(k)$: Assume $P(k+1)$: Relies on $P(k)$ OR Instead of the loop invariant, declare the algorithm sufficient for all graphs satisfying some condition K which is dependent on the size of the graph. For example, its diameter, or number of nodes, or path length....	Contradiction Statement of objects and their properties Assume the contradictory property Demonstrate contradiction Conclude contradictory property false
---	--

Proof tips

The structure of a brute-force proof	1. Establish loop invariant (path length k, using k vertices, maximal element $k...$) 2. Induction (as per usual) 3. Show that because of the size of the search space, the loop invariant covers the entire problem set. 4. Proof of termination
The structure of a greedy proof	1. Contradiction through swapping/comparing the optimal solution T and algorithm solution T^* 1. e.g., on the kth step, greedy does X. X must be the cheapest, hence, optimal is X. Or assume optimal not X, then greedy cheaper. 2. May be nested inside inductive proof, where contradiction used in the inductive step. 3. Proof of termination

Refer to pseudocode directly.

Disproofs:

To show that some algorithm does not work, a simple counterexample is required.

Note: some of the proofs below are rather unrigorous. But the main ideas should be there, and you can try formalising them yourself.

Examples

Palindrome Checker Algorithm

```
Function ISPalindrome(A)
// Input A an array A[1] is the first element
j ← 1
L1 ← A[j]
L2 ← A[length(A)-j + 1]
While (L1 equals L2) do
    j ← j + 1
    L1 ← A[j]
    L2 ← A[length(A)-j + 1]
end do
if (j equals length(A)) then
    return True
else
    return False
end if
end function
```

Consider all cases of input.

P(n): loop invariant -- J is equal to the number of palindromic elements (1-sided) in the first n elements of A.

Call the element length(a)-i+1 the "mirror" of element i.

Base case: 1 iteration

If the first and last elements are equal, j = 1, which is the number of palindromic elements so far.

Suppose P(k) is true

P(k+1):

Case 1: the k+1th element is not equal to its mirror. Then the number of palindromic elements encountered is k, and j is not incremented and the loop breaks.

Case 2: the k+1th element is palindromic, then j is incremented, and we have j+1 = k+1 the number of palindromic elements.

The loop breaks once $l1 > \text{length}(a)$ (i.e., $j = \text{length}(a) - 1$) or $l1 \neq l2$ as while loop increments j before completing actions on j. **Returning true if the palindromic elements are equal to all elements, and false otherwise.**

Insertion Sort

```
Algorithm INSERTION-SORT(A)
// Input A an array
for j ← 2 to length[A] do
    key ← A[j]
    // Insert A[j] into the sorted sequence A[1..j-1]
    i ← j - 1
    while i > 0 and A[i] > key do
        A[i + 1] ← A[i]
        i ← i - 1
    end do
    A[i + 1] ← key
end do
end algorithm
```

Give a proof of correctness by Induction – try the loop invariant idea

P(n): after the nth iteration of the j-loop, the first n elements will be correctly sorted relative to each other.

Base case: P(1): in the 1st iteration, key is the 2nd element and i is the first. If $A[i] > A[j]$, i is decremented until it is and the elements are swapped.

Otherwise, the inner loop ends, and nothing changes as $A[i+1] = \text{key}$. So the sorting is correct.

Note on the algorithm:

Let left denote the start of the list and right denote the end.

There is a duplicate element in position i and i+1 after an iteration of the while loop. After the kth iteration of the while loop, the k elements directly preceding key are shifted over by 1 position.

P(k+1)

All elements to the left of $A[k+1]$ are sorted. If $u = A[n] \geq A[k+1] \geq A[m] = v$ where $n, m < k+1$, then the elements $A[n]$ to $A[m]$ are moved to the right by 1. The new position of v is $A[m+1]$. The while loop fails for $A[n]$, and $n = m - 1$. so $i = m - 1$. The key is put into $A[m-1+1] = A[m]$, which is between k and u, which is correct.

A very simple algorithm LogUp that computes the rounded up integer value (ceiling) of the log base 2 of an integer x.

Function LogUp (x: integer):

```
1. i ← 0
2. y ← 1
3. IF x > 0 then:
4. WHILE y < x do:
5.   i ← i + 1
6.   y ← 2 * y
7. Return i
8. ELSE
9. Return "error"
```

Example Trace: LogUp(14)

x	Loop guard	i	y
14		0	1
	1 < 14	1	2
	2 < 14	2	4
	4 < 14	3	8
	8 < 14	4	16
	16 > 14	Stop loop	
Return 4			

P(n): returns ceiling of $\log_2 n$ for n

Loop invariant: after the ith iteration of the while loop, y is 2^i .

Base case: when $n = 1$, y is not $< x$, so $i = 0$ and is not incremented. This is correct as $\log_2(1) = 0$.

Assume P(k) is true so $i = \text{ceil}(\log_2(k))$.

P(k+1):

	i	y
k	i	2^i
Case 1: $k+1 \leq 2^i$	i	2^i
Case 2: $k+1 > 2^i$	i+1	$2^{(i+1)}$

if $k+1 > 2^i$, then $\log_2(k+1) > i = \text{ceil}(\log_2(k))$, so $\text{ceil}(\log_2(k+1)) > i$.

This only occurs when $k = 2^i$. $2^i \geq 1$, so $k+1 \leq 2^{i+1}$.

So the loop guard breaks on the next iteration, and i+1 is returned correctly.

Otherwise if $k+1 \leq 2^i$, then $\log_2(k) < \log_2(k+1) \leq i = \text{ceil}(\log_2(k))$. The loop guard breaks and i is returned correctly.

3. Consider the algorithm Reverse-Delete which finds the Minimum Spanning Tree of a connected, undirected, distinctly weighted graph as defined below. Prove by contradiction that it finds the Minimum Spanning Tree.

```
Algorithm Reverse-Delete(G)
// G is a connected, undirected, distinctly weighted graph
Let SE be the list of sorted edges in descending order of cost in graph G.
Foreach edge e in SE do
    If (deleting edge e disconnects G) then
        Do nothing as edge e is not part of a cycle
    Else
        Delete edge e from G
    End if
End do
```

Let G be the graph specified and T be the output. Suppose Reverse-Delete (RD) does not find a minimum spanning tree on G. Then:

Case 1: there is a cycle in T

If an edge that is a part of a cycle is considered by T, then it is deleted. As all the edges are considered, T must have no cycles.

Case 1.5: T is disconnected

Impossible because edges are deleted only if removing it doesn't disconnect G.

Case 2: the spanning tree found is not minimal.

Let T^* be the minimum spanning tree and T be the tree generated by RD. Then on some iteration, RD must delete an edge uv from T which is in T^* . uv must

	on some iteration, RD must delete an edge $u-v$ from T which is in T^*. $u-v$ must be a part of some cycle. As $u-v$ is the highest cost edge in this cycle, $T - u-v$.weight $\leq T^* -$ (other edge weight). If this process is repeated for every edge, we have $T \leq T^*$, which is a contradiction, so T must be the minimum spanning tree.
1. For the problem of finding the maximum element of an array of integers, the algorithm to find the solution findMax is shown below. Demonstrate the correctness of findMax using Proof by Contradiction. <pre> Function findMax(A) // INPUT: A[1..n] - an array of integers // OUTPUT: an element m of A such that $m \geq A[j]$, for all $1 \leq j \leq \text{length}(A)$ max = A[1] for j = 2 to A.length do if max < A[j] then max = A[j] end if end do return max End function </pre>	Suppose findMax does not return the maximum element of the list. Case 1: FindMax returns nothing. False as max defined to $a[1]$ and returned at the end. Case 2: FindMax returns a non-maximal element, m. Let the kth element, m^*, be the maximal element, and assume it is not returned. Every element is compared against the max. Induction: - Base case: for a list with 1 element, it must be maximal. - Suppose that returnMax works for all lists with k elements - On a list with k+1 elements, the maximal element of the beginning sublist with k elements is set to max (inductive hypothesis). If the k+1th element $>$ max, then the k+1th is maximal and max is set to it. Otherwise, max doesn't change. Both cases result in a correct output.

3.3 Applications

Sunday, 8 October 2023

6:14 PM

Key knowledge/skills	• characteristics and applicability of ADTs and algorithm design patterns • suitability of ADTs and algorithm design patterns for a variety of problem contexts • combinations of ADTs to meet complex problem requirements • the application of algorithms to answering real-world problems • describe how complex information can be represented by a combination of ADTs • select combinations of ADTs and algorithms that are fit for purpose • justify the suitability of ADTs and algorithm design patterns for particular problems • communicate the design of data models and algorithms • explain the interpretation of computed solutions in terms of their meaning to the original real-world problem being solved
----------------------	---

This unit covers a skill that will be used in a significant number of exam questions -- anything that asks for a data model/algorithm will require the following:

ADT

- The model (select ADT and outline how it maps to the problem at hand)
- Justification (application of graph algorithms, ease of access ... basically state any ADT feature)

Algorithm

- Design pattern (efficiency? optimality?)
- Input/output (how is the computed solution applied?)
- Communication (neat pseudocode? Consistent notation?)

How do you explain/describe/justify algorithms without pseudocode?

These are common things that will need to be discussed. Not all at once, and not for every question -- think of them as "helper functions" for your solution :)

Modify an existing algorithm	- Describe the aspect of the algorithm that will be modified • Floyd-Warshall outputs a distance matrix... • DFS marks nodes as visited... • Dijkstra randomly chooses a node if two have the same cost... - Describe a modification • If any cells are infinity... • $Visited \neq V$ • A tie-breaker could be...	
Describe a design pattern/ U4 algo	Brute force/generate and test	Brute force could be used... how? - A tree traversal, where nodes are recursively generated by changing [feature] from parent until [depth] reached. - Enumerate every combination of... by appending new elements to current solutions until the desired length is reached. (This is tree traversal!) - Divide and conquer by getting each possibility for subproblem, and generating combinations of subsolutions.
	Backtracking	Represent the problem as a decision tree/ state tree, incrementally adding [feature] to a parent node until [error] or [leaf node]. When a solution is found, [record/return].
	Tree traversal/ graph traversal	Used for backtracking and brute force. State the nature of

	decision tree or state graph being used.
Greedy	The [local best] can be chosen at each iteration - Lowest colour number - Cheapest edge
Decrease and conquer	Incrementally solving subproblem and reducing problem size
Divide and conquer	Split into [k] subproblems. The solution is [related to the subproblem]
Dynamic programming	A bottom-up approach by finding the optimal gain for A[1] through to A[final] by taking $\max(A[i], A[i-k] + A[k])$. - Prev. attribute and retracing the steps and summing the differences to obtain all components used. - Appending k.components to i-k.components
Hill climbing	Neighbouring solutions can be obtained by swapping one [attribute]. Repeatedly move to the neighbour with best [heuristic] until a maximum is reached.
Simulated annealing	Mention the following: - Initial solution - Generation of solutions - Acceptance criteria - Termination
A*	Repeatedly pick the unvisited node with lowest $f(n)$ value where $f(n) = c(n) + h(n)$. $C(n)$ is the cost to reach the node and $h(n)$ is a <i>predicted</i> cost to goal. Describe how $h(n)$ is calculated - Euclidean distance? - Closest bearing?
	In general, describing an algorithm is hard... clinching just <i>that</i> layer of precision without being too precise. Use functional programming paradigms to describe efficiently! - Get the maximum element vs "loop through and..." - Setting T to $\max(a,b)$... - Once k is obtained, retrace...
Justify ADT	- Describe a feature that maps to the problem • Directly models conflict... • Fixed size... • Allows emergency situations to be treated... - Suggest a future use • Graph traversal algorithms
Justify design pattern	- Feasible time (if not P) - Optimal solution (if P or if input size is small or if optimal substructure)
Interpret the solution	- How is the solution set stored? • Node attributes for shortest path • Matrix for FW • Appending solutions to a list if recursive • "Once [solution is obtained], retrace the steps/sum the rows/colours represent..." - How should people use the solutions?

4.1 Big O

Wednesday, 11 October 2023 2:00 PM

Key knowledge and skills	- the concept of classifying algorithms based on their time and space complexity with respect to their input - techniques for determining the time complexity of iterative algorithms - the definition of Big-O notation and its application to the worst-case time complexity analysis of algorithms - recurrence relations as a method of describing the time complexity of recursive algorithms - the Master Theorem for solving recurrence relations of the form: $T(n) = \begin{cases} a \cdot T\left(\frac{n}{b}\right) + kn^c & \text{if } n > 1 \\ d & \text{if } n = 1 \end{cases}$ where $a > 0, b > 1, c \geq 0, d \geq 0, k > 0$ - and its solution: $T(n) = \begin{cases} O(n^c) & \text{if } a < b^c \\ O(n^c \log(n)) & \text{if } a = b^c \\ O(n^{\log_b(a)}) & \text{if } a > b^c \end{cases}$ - examples and common features of algorithms that have time complexities of $O(1)$, $O(\log n)$, $O(n)$, $O(n \log n)$, $O(n^2)$, $O(n^3)$, $O(2^n)$ and $O(n!)$ - formally analyse the time efficiency of algorithms using Big-O notation - read off a recurrence relation for the running time of a recursive algorithm that can be solved by the Master Theorem or takes the form: $T(n) = \sum T(n - a_i) + b, \text{ where } a_i \in \mathbb{N}$ - use the stated Master Theorem to solve recurrence relations
--------------------------	---

Big O:

A function f is said to have a big O of g iff $f(n) \leq M \cdot g(n)$ for all sufficiently large values of n and M .
E.g. $99x^5$ has a big O of x^5 , it also has a big O of 2^x etc.

For graphs, time complexity is usually expressed in terms of V and E . Note that $|V|$ has a big O $|E|$.

In formal analysis, n is a function of the *bit-size* of input. See pseudopolynomial time.

It is bounded by E (but not tightly)

Description	How to solve	Example
Iterative algorithms with fixed for loops	Product of nested loop limits.	For i from 1 to n for j from 1 to n do stuff $O(n^2)$
Iterative with while loops or changing for loops	Depends on nature of iterators. Analyse carefully!	While condition is true Double length of list For element in list Do thing $O(2^n)$ where n is number of while iterations
Recursion decrease and conquer	$T(n) = \sum_{i=1}^d T(n - a_i) + b$ Approximately d^n	Return $f(n,1) + f(n-2)$ $O(2^n)$
Recursion divide and conquer	$T(n) = aT\left(\frac{n}{b}\right) + n^c$ Conditions: $a > 1$ (at least 1 subproblem) $b > 1$ (must be smaller) $c \geq 0$ (must be polynomial or polynomial limit)	$T(n) = T(n/2) + O(1) \rightarrow \log(n)$

	$\log b(a) < c \rightarrow n^c$, dominated by root $\log b(a) = c \rightarrow n^c \log(n)$, dominated by middle $\log b(a) > c \rightarrow n^{\log b(a)}$, dominated by leaves (lots of branch)	
--	--	--

Common time complexities

$O(1)$	Removing and inserting into a list Setting a variable Enqueue to queue
$O(\log(n))$	Binary search Divide and conquer algorithms with $c = 0$ Insertion into heap (pqueue)
$O(n)$	Finding max element of list Iterating through data type
$O(n^2)$	Sorting (bubblesort, insertion sort) Greedy best-first algorithms without priority queue
$O(n \log n)$	Sorting Divide and conquer with $c = 1$
$O(2^n)$	Checking all subsets
$O(m^n)$	Checking solutions where there are m possible options for n things
$O(n!)$	Permutations TSP

Interlude: pseudopolynomial time

Remember that n is a function of the bit length of the input? What does this suggest about algorithms where the input n is an integer? As an integer size n can be represented with $\log(n)$ bits, many seemingly polynomial time complexities of an integer n turn out to be exponential.

Examples:

- Prime checking takes $\sqrt{n}$ actions $\leftrightarrow \sqrt{2^{\text{bits}}}$
- Dynamic solutions to coin change and knapsack are pseudopolynomial with $O(nW)$, W is number to make and n is number of coins/items

4.1 Complexity classes

Thursday, 12 October 2023 12:22 PM

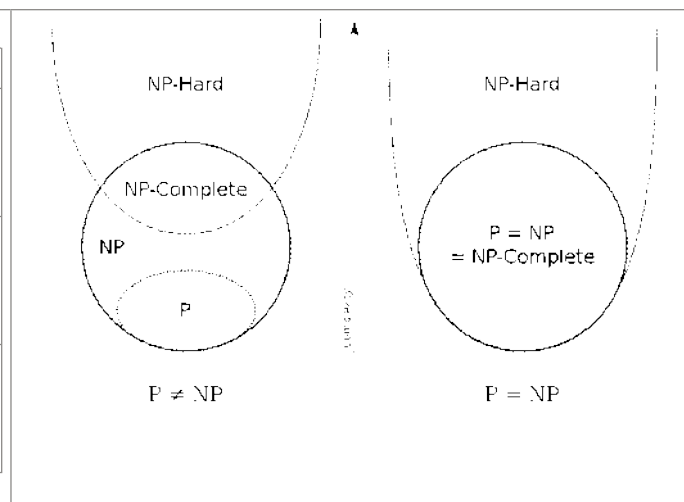
Key knowledge/skills	• the concept of the P, NP, NP-Hard and NP-Complete time complexity classes for problems • consequences of combinatorial explosions and indicators for them • the feasibility of NP-Hard problems in real-world contexts • demonstrate how exponentially sized search and solution spaces impose practical limits on computability • evaluate the suitability of algorithms to particular contexts based on their time or space complexity • estimate the time complexity of an algorithm by recognising features that are common to algorithms with particular time complexities • describe characteristics of problems in the P, NP, NP-Hard or NP-Complete time complexity classes, including the consequences for a problem's feasibility of it belonging to one of these classes
----------------------	---

What are decision problems?

- The set of problems to which the output is some boolean. Typically "yes" or "no" is expected.
- The Halting Problem is a decision problem. So are all the NP-Complete problems.
- In mathematics, we can formalise this with sets. The problem is to decide whether i is in a set S that fulfills some condition.
 - o $S = \{i \mid i \text{ has a Hamiltonian Circuit}\}$.
 - o $S = \{i \mid i \text{ fulfills condition}\}$
- **We care about these because they help define the NP class.**

Complexity classes:

P	Solvable in polynomial time.
NP	Decision problems for which "yes" can be verified in polynomial time. It includes P as a subset, as any problem solvable in p-time is also verifiable (by resolving the problem!)
NP-hard	At least as hard as the hardest NP problem. NP problems can be reduced to NPH problems in polynomial time -- i.e., NPH's solution can be used to solve NP.
NP-complete	The union of NP-complete and NP-hard. Verifiable in p-time when the output is "yes" AND transformable from all NP problems. - Must be a decision problem



These complexity classes are based on the Turing Machine. A deterministic TM relies only on its current state and input to make a decision as the state, **as the state-symbol pair maps one-to-one to an action.** Whereas a non-deterministic one may have multiple actions for each state-symbol pair, resulting in a "computation tree" --NTMs are just thought experiments.

Every polynomial time program on an NTM can be mapped to a non-polynomial time algorithm on a DTM.

P=NP?

If $P = NP$, then every problem verifiable in P-time is also solvable in P-time. This also means every problem can be mapped in polynomial time to every other problem, as solving is only polynomial.

Cobham's thesis:

- Computational problems are feasible to solve only if they can be computed in polynomial time due to the rapid growth of solution spaces. Hence, only problems in P are considered to be tractable.
- The computational complexity class of a problem is independent of the model of computation used. (This is otherwise known as the extended Church-Turing thesis)

Limitations of this thesis:

- Ignores all lower order and constant terms,
- Ignores size of exponent for p-time algorithms
- Ignores typical input size

Nonetheless, it is always good to reference it when speaking of *infeasibility*. An exponentially (or worse) growing problem will always prove impractical to solve due to time/resource constraints because big inputs require really big resource allocations.

Estimating complexities and combinatorical explosions:

A **combinatorial explosion** occurs when the solution space of a problem grows intractably due to its combinatorial underpinnings. Examples such as solving Latin Squares, or TSP, are NP which indicated by the "explosion" of possible solutions.

If asked to estimate/justify the tractability of such a problem, mention the *number of combinations* and explicitly include combinatorics!

Example from Latin Squares:	
<i>n</i>	The number of Latin squares of order <i>n</i>
1	1
2	2
3	12
4	576
5	161,280
6	812,851,200
7	61,479,419,904,000
8	108,776,032,459,082,956,800
9	5,524,751,496,156,892,842,531,225,600
10	9,982,437,658,213,039,871,725,064,756,920,320,000
11	776,966,836,171,770,144,107,444,346,734,230,682,311,065,600,000

Example from CHES3:

```

Algorithm payUs(fee, hours)
//NOTE: max_pay is a global variable initially set to zero
  if(hours > 0) then
    for i in 1 to hours do
      thisfee = fee[i] + payUs(fee, hours - i)
      if (thisfee) > max_pay then
        max_pay = thisfee
      end if
    end do
  end if
  return max_pay
end algorithm

```

The call tree generated by this function essentially counts all compositions of n , as it is subtracting $1, 2, \dots, i$ each time and creating a new part.

Hence, $O(2^{n-1}) \rightarrow O(2^n)$.

Alternatively, drawing a call tree and observing that each node has a number of children can lead to straight-forward observations too. Recursive Fibonacci has order K^n , $1 < K < 2$. This is because each node calls 2 child nodes, but base cases occur at different stages.

Interlude: problem types

We talk about decision problems, and optimisation problems, etc. etc. without many formal definitions. For example, a decision problem requires a decision output. What a surprise!

Turns out a bit of systematic summary ties things together a bit nicely.

Search problems	A very (very!) broad class of problems. Asks whether some structure A can be found within some structure B. - Search the space of all possible circuits to find the lowest cost Hamiltonian Circuit - Search graph B for node A Search problems can also be expressed in terms of other problem classes. For example, the decision problem "is A in B?". Or the optimisation problem where the solution space is interpreted as a searchable object.
Decision problems	Problems which require a yes/no solution. - Is there a solution to $x^2 = -1$ in the reals? - Does a knapsack configuration exist with $\text{cost} > C$ and $\text{weight} < w$? Decision problems define the P and NP classes. Search problems in such classes, like finding an MST, have the decision versions reducible to the search version. Because if you can find it, you can decide whether you've found it.

Optimisation problems	The goal of an optimisation problem is to minimise the loss function or to maximise the reward function. The loss function could be - Single source shortest path: the sum of the distances from s to every other node. - Maximum margin classifier: width of the margin - Mean squared error: neural network average error
-----------------------	---

Reducibility

Problem A is said to be reducible to a problem B if the solution B can be used to obtain the solution to A in polynomial time .

4.2 Non-heuristic design patterns

Thursday, 19 October 2023 7:19 PM

Key knowledge and skills	• the binary search algorithm (decrease or divide?) • divide and conquer algorithms that have linear time divide and merge steps, including mergesort and quicksort • dynamic programming algorithms that require no more than a single dimension array for storage, including the Fibonacci numbers and change-making problem • tree search by backtracking and its applications apply the divide and conquer, dynamic programming and backtracking design patterns to design algorithms and recognise their usage within given algorithms
--------------------------	---

All these paradigms aim to find an optimal solution.

Backtracking

- *Refinement of brute-force approach*
- Invalid candidates are discarded, search tree is pruned.
- Note that there are two uses of "backtrack"
 - To backtrack, i.e., to revisit previous nodes as in DFS/BFS
 - Backtracking paradigm, i.e., backtrack when invalid partial candidate

Tree search refers to the search of a [tree structure](#), whether implicit or explicit. When VCAA asks for tree search, always define the tree being searched in terms of nodes and edges, then link it to backtracking by pruning branches.

Sample answer from VCAA 2017	Brute force could be used. A tree traversal could be used where each node represents a semi-complete permutation of the word. Each node would have a child connected by an edge to the words that could be made by changing the next vowel in the word. This repeats until all words possible are generated and the leaves (words with all vowels marked) can be taken as output.
------------------------------	---

Divide and conquer (see: decrease and conquer)

- *Non-overlapping subproblems*
- Break problem into smaller subproblems of the same type until trivial
- Each subproblem is a fraction of the original problem
- Combine sub-solutions to form optimal solution

Binary search	Binary search is regarded by some as divide and conquer, due to geometric reductions in input size. By others, it is regarded as decrease and conquer as only one subproblem is obtained -- some of the input is discarded. By strict definitions, it is neither. It neither uses >2 subsolutions to form the entire solution, nor
---------------	---

Dynamic programming

- *Overlapping subproblems*
- Optimal substructure
- Build up solution by solving overlapping subproblems only once.
- Bottom-up design or top-down memoization.

Formalisation	Optimal substructure - optimal solution can be constructed from optimal solution to subproblems - E.g., shortest path problem, longest increasing subsequence problem - This yields more efficient performances by greedy/divide and conquer/dynamic methods The Bellman Equation - "long term gain" is equal to "current gain" plus "expected future gain". Valid for all
---------------	--

dynamic programming problems and most obviously seen via Dijkstra's algorithm.
 - Another way to think about it is that the optimal solution to a subproblem isn't dependent on the optima of a superproblem. In change-making, making \$10 has no effect on making \$4.

Note: formalisms not required in VCE algorithmics.

Change-making problem

A specific instance of the knapsack problem, where there is an infinite amount of each item and the goal is to make the weight with the fewest number of items. Pseudo-polynomial with respect to number of items/size of input with $O(nW)$ for a dynamic programming solution n = item types, W = total cost necessary.

Coin(denom, goal):

A = new array size goal

Set all elements of A to infinity

For item in denom:

A[item] = 1

For i from 1 to length(A):

For j from 1 to length(denom):

tentative = i - denom[j]

if A[i] > tentative

A[i] = tentative

Algorithms

Binary search:

BinarySearch(ls, lo, hi, goal):

mid = floor((lo + hi)/2)

if ls[mid] = goal

return mid

else if ls[mid] > goal

return binarySearch(ls, lo, mid-1, goal)

else

return binarySearch(ls, mid+1, hi, goal)

Proof:

Loop invariant: $lo \leq \text{goal} \leq hi$ after every recursive call for some list L

P(n): loop invariant holds for L after the nth call

P(1): lo and hi are the first and last elements of L, so every element must be either lo, hi, or between them

P(n+1):

Mid is taken as $(lo+hi)/2$ which is the midpoint of lo and hi

Case 0: mid is L in which case correct result returned.

Case 1: goal > L[mid]. Then goal is after mid in the list. BinarySearch is called with (L, mid, hi, goal) which sets lo to mid, so goal must be between lo and hi.

Case 2: goal < L[mid]. Then the goal is between mid and lo. BinarySearch called with (L, lo, mid, goal) which sets hi to mid so goal must still be between lo and hi.

Loop invariant proved.

As the boundaries lo and high are strictly narrowing, we eventually converge on the goal

4.2 Heuristics and NP problems

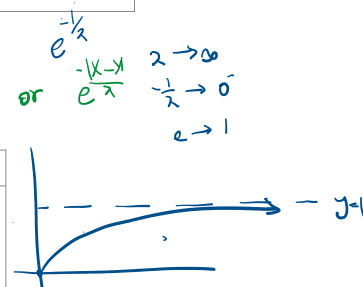
Thursday, 19 October 2023 7:25 PM

Key knowledge and skills	- the application of heuristics and randomised search to overcoming the soft limits of computation, including the limitations of these methods - hill climbing on heuristic functions, the A* algorithm and the simulated annealing algorithm - the graph colouring, 0-1 knapsack and travelling salesman problems and heuristic methods for solving them - develop different algorithms for solving the same problem, using different algorithm design patterns, and compare their suitability for a particular application - apply heuristics methods to design algorithms to solve computationally hard problems - explain the application of heuristics and randomised search approaches to intractable problems, including the graph colouring, 0-1 knapsack and travelling salesman problems
--------------------------	---

Note that the algorithms stated here are more so design patterns in themselves than explicit algorithms. I guess they're specific enough to qualify as algorithms. But implementation may vary wildly, unlike many U3 algos which are more specific and more self-contained.

Hill climbing	Pick the best neighbour with highest gradient to move to with each step. May get stuck in local minima. Like best-first search without memory. Repeat until no better neighbours.
A*	Use heuristic $h(n) = f(n) + g(n)$ to enqueue neighbours into priority queue for graph traversal. $f(n)$ is the cost/distance to get to the node, whereas $g(n)$ is the heuristic function.
Simulated annealing	Metaheuristic. (I.e., heuristic for using heuristics), like random search but with a chance to accept worse solutions to escape local minima. Uses the probability distribution $e^{-1/x}$ most of the time for determining chance of accepting worse solutions, where x is the temperature. May account for "badness" of problem by using $ X-Y $.
Randomised search	Sample neighbouring solutions randomly until a better solution is found, then move to it. Repeat until termination criterion reached.

Note that many of these approaches are *inherently greedy*; *greediness is a heuristic in and of itself*. With the exception of SimAnneal, they always pick the best local choice.



NP complete problems

	Optimal	Decision	Applications
Graph-colouring	Determine the minimum number of colours needed to colour a graph such that no two neighbours have the same colour.	Determine if it is possible to colour a graph with k colours such that no two neighbours have the same colour.	- Conflict management • Timetables • Storing hazardous goods • Seating people at tables - Contrived artworks - Modelling physical things • Chemical structures
Knapsack	For a collection of items with a cost and weight, determine the selection of items which maximises cost while remaining under a weight threshold.	Determine if some weight/cost requirement for a collection of items is possible.	- Scheduling jobs/tasks. - Robbery, travelling, sales
Traveling salesperson	Find the cheapest Hamiltonian path/circuit on a graph.	Determine if a Hamiltonian path/circuit under some distance threshold exists.	- Any real-life travelling problem - (From Georgia) surgery. I.e., modelling incision zones as a graph. - Window cleaning on high buildings, Imao

Heuristics and other methods for solving NP-complete problems. Example solutions.

	Heuristics	Greedy	Optimal
Graph colouring	Neighbours can be generated through colour swaps of a few nodes for simulated annealing. Heuristic could be "number of conflicts" + "number of colours used" or a combination thereof.	Colouring all the neighbours of a node through BFS/DFS, using the minimum number of colours locally by iterating through the possible colours 1,2,3... until an acceptable one is found.	Backtracking through tree traversal where each node is a partially coloured graph, and any graphs with conflicts are pruned from the search space. Each node would have child nodes connected via an edge, where the child nodes are created by considering all possible colourings for the next node. Total number of colours is total number of vertices. $O(V^V)$

Knapsack	Neighbours can be generated through item swap/adds, representing in and out as vector of booleans. Simulated annealing or hill climbing, etc.	Rank items in order of best cost/weight ratio, iteratively selecting items from the top of the queue which can fit.	Backtracking through tree traversal, each node is vector of booleans starting from $[0,0,\dots,0]$ where 0s are not in the sack and 1s are. Child nodes generated by appending either 0 or 1 until the required depth is reached. Any sacks which go over the weight threshold are discarded. Dynamic programming can be used for a pseudopolynomial algorithm!!
TSP	Neighbours can be generated through rearrangement of nodes, and not discarded if such a rearrangement doesn't represent a path/circuit.	Nearest neighbour heuristic. Always pick the nearest neighbour until path created. May not result in solution. Greedy rank edges.	Search tree with each node as a partial circuit, and child nodes are created for all possible next moves, being the parent node with an extra edge attached. Or just generate all permutations of nodes and check if valid. (You can also formulate this in terms of tree generation, where child nodes are just any node not in the parent path) $O(n!)$

4.3 Origins of comp sci

Wednesday, 18 October 2023 6:24 PM

Key knowledge/skills	• the historical connections between the foundational crisis of mathematics in the early 20th century and the origin of computer science, including Hilbert and Ackermann's Entscheidungsproblem and its resolution by Church and Turing • characteristics of a Turing machine • the concept of decidability and the Halting Problem as an example of an undecidable problem • implications of undecidability for the limits of computation • explain the historical context for the emergence of computer science as a field • describe the general structure of a Turing machine • demonstrate the existence of hard limits of computability using the Halting Problem
----------------------	--

Hilbert's Program was an attempt driven by German mathematician David Hilbert in the 20th century to formalise the entirety of mathematics, driven by the foundational crises of mathematics:

- Russell's paradox of set theory. Set that contains all sets except itself.
- Parallel postulate cannot be proved
- Theorems of arithmetic unprovable from Peano arithmetic

Some goals of the program:

1. A formulation of all mathematics. I.e., precise formal notation and well-defined rules.
2. **Completeness. Every true statement should be provable in the formalism.**
3. **Consistency. No contradiction should be able to be derived using the rules of the formalism.**
4. Conservation. All statements about "ideal objects" (e.g., infinite objects) should be able to be proved using only "real objects"
5. **Decidability. There should be an algorithm (effective way of computation) for deciding the truth of any statement.**

Decidability forms the basis of Ackermann and Hilbert's **Entscheidungsproblem** (Decision Problem) which asks for a way to effectively compute the truth of any statement in a formal system.

This was the motivation behind Church and Turing's computational systems, which were formalisms of what "effective computation" meant.

The proof of the undecidability of the Halting Problem showed the undecidability of mathematics unto itself. There are some things that cannot be computed in the way we understand computation. For example, the general Collatz Conjecture was shown to be undecidable. The undecidability of the halting problem means that an automated algorithm verification system is impossible.

The Church-turing Thesis:

Defines computability based on the abstract notion of Turing Machine computable, independently of any physical constraints.

The STRONG CT thesis: (not in sd)

Tractable problems are tractable iff they are tractable on a turing machine.

Turing machines:

- Unbounded tape of discrete cells
- Table of symbols
- Table of states
- Read/write head can move across cells

Lambda Calculus:

Recursive system developed by Alonzo Church. An equivalent system of computation (in terms of efficiency, computability) and also defines computability by the Church-Turing thesis.

- Read/write head can move across cells
- Transition function

Turing machines can be formalised by a 7-tuple.

L, Finite, non-empty set alphabet symbols
b in L, the blank symbol. The only symbol of which there can be an infinite number
S in L, set of input symbols allowed to appear in initial tape contents
Q, a finite non-empty set of states
q ₀ , the initial state
F in Q, final/halting states
d: (Q\F) × L → Q × L × {L,R}. State x symbol → state x symbol x movement.

This is how Turing described it:

"...an unlimited memory capacity obtained in the form of an infinite tape marked out into squares, on each of which a symbol could be printed. At any moment there is one symbol in the machine; it is called the scanned symbol. The machine can alter the scanned symbol, and its behavior is in part determined by that symbol, but the symbols on the tape elsewhere do not affect the behavior of the machine. However, the tape can be moved back and forth through the machine, this being one of the elementary operations of the machine. Any symbol on the tape may therefore eventually have an innings."
—Turing 1948, p. 3

From https://en.wikipedia.org/wiki/Turing_machine

The halting problem:

Is there an algorithm that can determine in advance whether any Turing Machine with ruleset P halts on input i?

- More generally, given *any program P* and input I, does P halt on I?

Alan Turing independently published his proof in 1937. A year before, Church had published a proof for the undecidability of a problem in lambda calculus.

Halt(P,I)	Halt if P halts on I
	Loop if P loops on I
Perverse(P,I)	Halt if P loops on I
	Loop if P halts on I

The non-rigorous proof of concept relies on the construction:

- Perverse(Perverse, Perverse)
- If P halts on P, P doesn't halt on P.... etc.

However, note that we're only giving one input into the nested Perverse whereas the actual function requires two inputs.

Turing's proof was more formal, and based on Cantor's diagonalisation for countable and uncountable infinities. I haven't added it here because explaining that in a VCE exam is just a waste of time, but do find it and read it.

4.3 AI workings

Thursday, 12 October 2023 12:22 PM

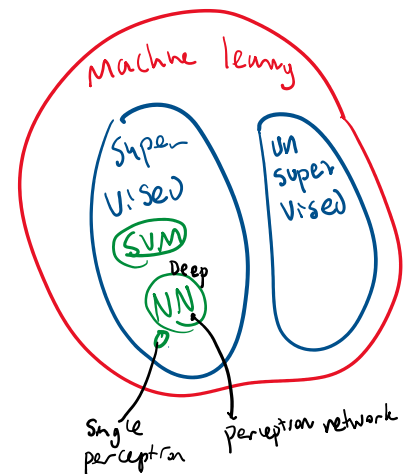
Key knowledge/skills	- the concept of training algorithms using data - the concepts of model overfitting and underfitting - support vector machines (SVM) as margin-maximising linear classifiers, including: - the geometric interpretation of applying SVM binary classification to one- or two-dimensional data - the creation of a second feature from one-dimensional data to allow linear classification - neural networks, including: - the structure of multi-layer perceptron neural networks - the evaluation of outputs using forward propagation - training neural networks by using iterative improvement of the edge weights to reduce the output error - explain, at a high level, how data-driven algorithms can learn from data - explain the optimisation objectives for training SVM and neural network binary classifiers - explain how higher dimensional data can be created to allow for linear classification - describe the structure of a multi-layer perceptron neural network - evaluate the output of a small multi-layer perceptron neural network using forward propagation - explain the consequences of model overfitting or underfitting
----------------------	---

Let's start with some definitions.

AI:

Artificial intelligence is an umbrella term for automated decision making which is the intelligence demonstrated by machines. Used to describe an agent that can perceive its "environment" and adapt to maximise chances of reaching a goal.

Traditional algorithms	Machine learning.
Not much to say. What is an algorithm? An unambiguous set of steps!	Machine learning describes a branch of artificial intelligence. Describes machines discovering their own heuristics/algorithms by being trained on data and iteratively improving based on feedback.
Input algorithm and data. Output results.	Do action. Check if action good or bad. Do new action if less bad. Repeat. Input input and results. Output heuristic/data-based algorithm.
Used for problems which are easily characterised in terms of input/output relations. E.g., shortest path.	Used when the explicit relationship between input data and outputs is unknown. E.g., facial recognition. Hard to quantify.



Overfitting	Underfitting	Supervised machine learning refers to a training process where a relation is formed between inputs and desired outputs by labelling the training data with actual outputs.
Works well on training data Learnt insignificant variations in training data, rather than overarching patterns	Doesn't learn rule at all. Works poorly on both training and testing data.	

Loss function:

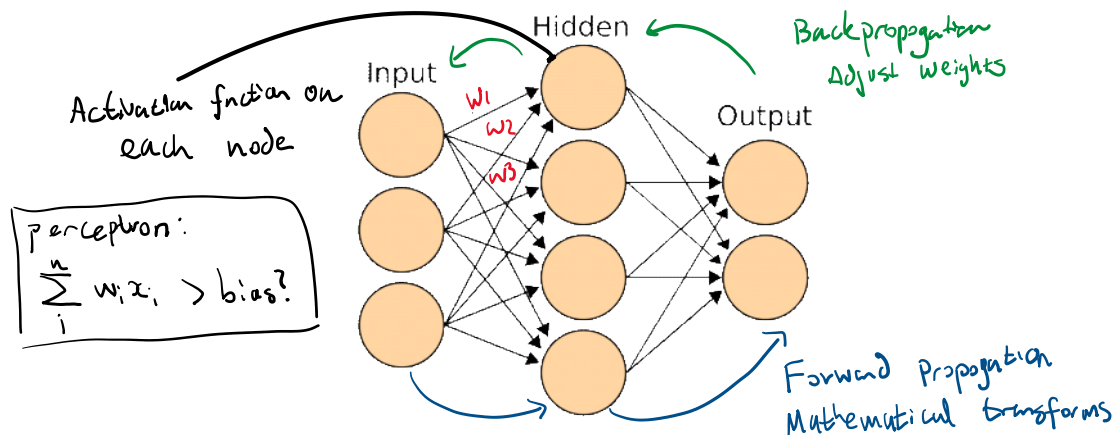
Loss/cost functions map events onto some representation of "cost". The minimisation of this function is the goal of both NNs and SVMs, so AI training can be seen as an *optimisation problem*.

Neural network:

A network of artificial neurons where the outputs of neurons in previous layers are fed as inputs to neurons in next layers, a process known as forward propagation. These neurons are often referred to as nodes in a neural network.

NNs are characterised by information flow. Feed-forward NNs only allow uni-directional flow input->hidden->output. Recurrent NNs may have loops and cycles. *VCE algo only deals with feed-forward networks.*

Neuron	Simple decision making component that applies a mathematical function called the activation function.
Forward propagation	When outputs from layers are fed forward into proceeding layers for further transformation.
Back propagation	When the loss is fed backwards and edge weights are adjusted based on feedback.
Epoch	one forward and backward pass



Optimisation objective:

Common loss functions:

- Mean-squared error. Average of squares of errors
- Standard deviation.

Activation functions:

An activation function is a function that calculates the output of a neuron. Common ones include:

- Linear: $f(x) = ax+b$
- ReLU: $f(x) = 0$ if $x \leq 0$, else x .
- Logistic: $f(x) = (1+e^{-x})^{-1}$

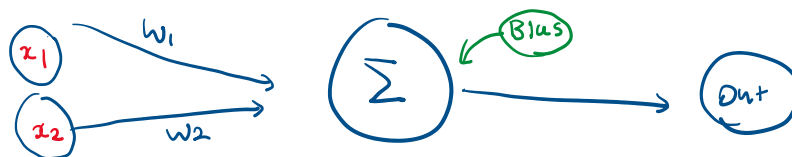
The identity activation function is such that $f(x) = x$... not much use most of the time...

The perceptron activation function (Heaviside step function)

Perceptrons are linear binary classifiers. Their activation function can be expressed as a piecewise on the weighted sum of inputs.

- $f(x) = 0$ if $\text{sum}(wx) + \text{bias} < 0$, else 1.

A multilayer perceptron network is a specific type of NN that only has perceptrons for its neurons.



Universal approximation theorems

https://en.wikipedia.org/wiki/Universal_approximation_theorem#Arbitrary-depth_case

- An arbitrary depth NN with the **ReLU activation** function can approximate any "Lebesgue integral function". (Lebesgue integration is the extension of integration to higher dimensions and wacky analysis magic).
- A perceptron network with an unbounded width can approximate the step function, can approximate sigmoidal functions, can approximate anything!

In relation to the course, all this tells us is that any function can be approximated by NNs including multilayer perceptrons, so a **complex-enough NN can be used to classify anything.**

SVMs

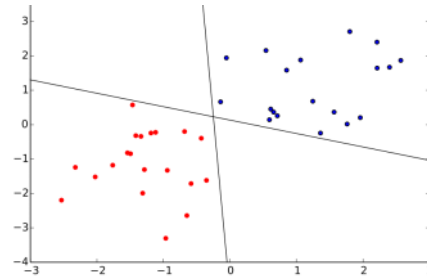
Actually 2d SVMs can be seen as an optimised perceptron!

In the 1987 paper ("Learning algorithms with optimal stability in



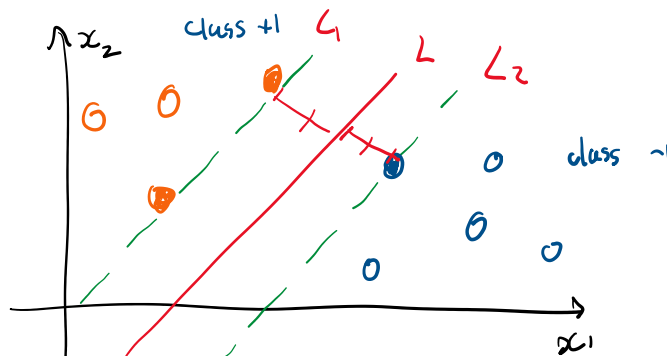
neural networks"), *The Perceptron of Optimal Stability* was designed to solve the problem where many possible linear classifiers are possible. Like the image to the right.

Though a perceptron can guarantee a classification, it cannot guarantee a "best classification", so a poorly done classifying line may need to be fed a lot of data before reaching an acceptable level. Stability refers to not needing to be adjusted so much.



A support vector machine is a linear binary classifier. However, the kernel transform allows the creation of an additional feature as a function of existing features. This adds a dimension and classification therein can be linear even if the original data was not linearly separable.

Support vectors	Closest data points to the hyperplane which define the hyperplane and margin
Hyperplane	Decision boundary of the form $ax_1 + bx_2 + \dots + b = 0$
Margin	Distance between support vector and hyperplane - Hard: no misclassifications allowed - Soft: compromise between margin width and classification accuracy
Kernel	The function used on current features to create a new feature



Optimisation objectives:

Typically, the hinge loss function is used for SVMs.

The hinge loss function maximises the classification margin.

What the hell is gradient descent?

	Mathematical idea	NN pseudocode
1	Start at any point on the function	Randomise weights of NN
2	Calculate slope of loss	Loss = $ \text{activation}(\text{input}, \text{weight}, \text{bias}) - \text{actual} $
3	Take a step in the negative direction of slope to reduce loss	$\text{weight}[i] = \text{weight}[i] + \text{loss} * \text{error} * \text{input}[i]$
4	Repeat until minimum reached	Repeat until convergence

Limitations and advantages of AI

	Good	Bad/limitations
SVM	- Handles many features well - Kernel transform allows separation - Maximum margin classifier -- good for classification when distinct classes are present	- Poor performance with many datapoints - No probability estimates for "goodness" of guess - Kernel transform may be hard
NN	- Universal approximation theorem - Large datasets can improve precision of algorithm	- Overfitting - Not a maximum margin classifier -- less stable - Needs a lot of training data to iteratively improve

4.3 AI philosophy/history

Wednesday, 18 October 2023 6:24 PM

Key knowledge and skills	• philosophical conceptions of artificial intelligence, including the Turing Test, weak AI and strong AI • Searle's Chinese Room Argument, including standard responses both for and against • the factors leading to a resurgence in neural networks in the late 20th century • describe and compare the Turing Test, strong AI and weak AI as conceptions of artificial intelligence • describe the Chinese Room Argument and mount an argument for or against it
--------------------------	---

A timeline of AI

1944	Artificial neuron (McCulloch and Pitts)
1950	Turing Test
1957	Perceptron demonstrated
1969	Lack of computational power: AI winter begins
1960s	Expert systems (can be seen as very early examples of training data)
1980	Backpropagation and edge weight modification sparks start of NN usefulness, and 2-3 layer NNs are used. Searle's Chinese room published
Late 1980s	NNs considered too complex and computationally expensive again
1990	SVM popularity surges
1991	World wide web provides ample data for later training NNs
1997	Deep Blue beats Garry Kasparov (Noam Chomsky was not impressed: "tractor beats man at weightlifting" he said)
2010s	Resurgence in NN popularity as computational power begins to match theoretical ability. (Moore's law). Begin of deep learning proper. Partially credited to computer game industry as the parallel processing power of GPUs was partially driven by the demand of basement-dwelling gamers.

The Turing Test

A human judge tries to distinguish between AI and human responses. It measures AI's language processing ability. An AI is said to have passed if the judge fails half the time.

Language/behaviour as a marker of intelligence. Congruent with otherminds argument.

Strong and weak AI

While strong AI and weak AI are broad descriptors of "types" of AI, the terms were coined by Searle for his Chinese Room argument. His definitions are what is known as the "strong AI hypothesis".

The strong AI hypothesis: "...computers given the right programs can be literally said to understand and have other cognitive states." (Searle 1980)

Modern AI research is only concerned with the attainability of AGI. The consciousness of the machine is considered irrelevant. Hence, only the weak AI hypothesis is necessary: "can AI act like it has a mind and consciousness?".

	Weak AI	Strong AI
Searle's definition	Only acts like it has a mind and consciousness	Has a mind and consciousness.
General usage	Designed to perform specific tasks - "Narrow AI" - Narrow range of behaviours - Heavily data driven as it can only imitate - Requires human input and feedback	Intelligent like a human - "Artificial general intelligence" - Self-aware and sentient - Learn a broad range of activities - Match/surpass all human/animal capabilities

Relation to Chinese Room:

- Searle argues that Strong AI is impossible.
- Does not necessarily argue against AGI, as AGI and strong AI are usually considered non-synonymous: *consciousness is not a pre-req for the appearance of consciousness*.

Relation to Turing Test:

- Poor test for AGI: only tests language processing. No test for learning/breadth.
- Poor test for strong AI: semblance of consciousness
- Ok test for certain aspects of weak AI: language processing

The Chinese Room thought experiment

- Syntactical processing != semantic understanding.
- A man in a chinese room who blindly follows rules does not have an understanding of chinese. Just rules.
- No personal intent

Broadly speaking, this can be seen as an offshoot of the mind/body argument.

Systems	While the man inside the room doesn't understand his responses, the system, as a whole, does. I.e., the combined component of the rulebooks/etc. possesses a semantic understanding. For example, how would the man answer "what is your opinion on Mao Zedong?". No matter what, the creators of the rules must have understood the question and are components in this system. Searle's response: If the man memorises all the rules, he still comprises the entire system.
Robots	The man in the room cannot link his utterances to real-world experiences. If hooked up to sensory inputs, the man could begin to gain semantic understanding. Searle's response: Treat all inputs as <i>just inputs</i>. Input-output pairs not contingent on the type of input!
Neural/brain	Brains are complex, and organic matter is no different to electric matter. A complex enough machine could simulate a human brain! Every single neuron firing simulated perfectly by the AI would be identical. What if neurons in a human were replaced one by one? Searle's response: Does not simulate intention. Searle argues that mental phenomena are dependent on the biochemical reactions that occur. Instead of imagining the system with electric stuff, imagine if it were just water and pipes. Is that conscious? Or is that just a simulation? Understanding attributed to electricity stuff is just due to obfuscation. In relation to NNs, Searle argues that there is no cognition, intent, mental states.
Otherminds	The semblance of intelligence is enough. E.g., you can't verify the consciousness of anything, animal or human or machine. Searle's response: Still just a simulation. No traits of strong AI.

4.3 AI Ethics

Friday, 27 October 2023

11:22 AM

Key knowledge/skills	ethical issues related to artificial intelligence and data-driven algorithms, including transparency , accountability , bias and machine ethics explain and discuss ethical issues related to artificial intelligence and data-driven algorithm
----------------------	--

The use of AI has ethical repercussions.

Classify them into 2 groups

- Validity/correctness (immediate effect)
 - o Has the data been manipulated to benefit a certain group or individual?
 - o Is the AI biased?
 - o Are outcomes explainable by humans?
- Long-term effect
 - o Who has access to AI data?
 - o Does the use of AI infringe upon human dignity or privacy?
 - o Jobs?

	Ethical considerations	AI
Privacy and surveillance	- The right to be let alone - Privacy as an aspect of dignity -- human rights - Data security	• Powerful processing of individual data infringes privacy: stalkerish! • Can reveal information we don't know about ourselves • Leads to manipulation
Manipulation	- Autonomy of individuals - Corporation profit at individual expense - Political manipulation - Deepfakes - Manipulation of AI systems	• Use of known weaknesses to target individuals • Threat to democracy • AI capabilities can generate convincing false narratives • Addiction/procrastination generation
Transparency	- Explanability and verifiability and honesty - Stakeholder informed choices	• Can't verify complex systems against bias or guard against manipulation effectively • Threat of "algocracy"
Bias	- Unfair prejudices and discrimination - Historical bias, confirmation bias - Power to individuals - Access to AI	• AI replicates bias in training data, perpetuating historical bias • Predictions take power away from individuals and self-determination • WEIRD: white, educated, industrialised, rich, democratic.
Accountability	- Autonomy as a basis for responsibility - Split accountability	• Errors caused by AI may entail compensation. • Should autonomous systems be held responsible? How? • Copyright and plagiarism
Dignity	- Employment - Human interaction	• Distribution of AI-created wealth • Risk of impersonal relationships with others and a lack of care • Dehumanising
Machine ethics	- Machines as subjects - Agency: ethical productivity and receptivity - Singularity	• AI status as legal entity • Sentience and moral obligation to not harm sentient beings • Reciprocal relationship between responsibility and rights • AI is duplicable

When to use what?

	SVM	NN
Pros	• More explainable with obvious geometric meaning • Suitable when classes are very distinct • Less data needed to train	• Greater levels of precision by considering every datapoint • Can model any function

Field	Issue snapshot	Fill-in-the-blank for more potential issues :)
AI artwork	- Plagiarism of human content, non-consensual training - Deepfakes and manipulation - Biases reflected by prompts	
Surveillance systems	- Bias in crime detection due to training data - Breaches of privacy - Manipulation for ad placement	
Healthcare	- Accountability for error: "retribution gap" - Intransparency leads to non-informed choices - Lack of human interaction	
Self-driving cars	- Machine ethics striking a balance between individual utility and safety - Accountability for accidents	
Autonomous weapons	- Cheapening of human life and dignity - Accountability for war crimes - Machine ethics and a responsibility to Not Harm.	
Social media	- Privacy of personal browsing habits and trading of personal data - Profiling of individuals - Manipulation for addiction generation and politics - Hard to enforce responsibility	
Job/scholarship screening	- Historical biases perpetuated leads to unfair results - Training may infringe on privacy and data security - Machine ethics: "greater good" or fairness? - Evaluation by automated system can be dehumanising	
Student grading	- Accountability for unfair grading - Takes away from student-teacher interaction - Prone to manipulation - Weaknesses in AI may be exploited to detriment of learning	